AUDITORIA, INTEGRIDADE E DETECÇÃO DE ALTERAÇÕES

*A pergunta do cliente era simples e ninguém soube responder: **"esse servidor ainda é o que vocês instalaram?"** Quatro anos no ar, oito administradores, correções de madrugada e scripts baixados da internet rodados como `root`. `/usr/bin` tinha 2.400 arquivos — e nenhum ponto de comparação.*

089
GUIAS LINUX

O PIPELINE: SÓ A ETAPA 1 NÃO PODE SER FEITA DEPOIS

1 — LINHA DE BASE
Hash de cada arquivo **no momento em que o sistema estava íntegro**.

→

2 — COLETA
Estado atual: hash, dono, permissão, `mtime`, tamanho.

→

3 — COMPARAÇÃO
O que mudou, o que sumiu, o que apareceu do nada.

→

4 — TRIAGEM
Mudança esperada — ou **alerta**? É aqui que o esforço sobrevive ou morre.

CUSTÓDIA EXTERNA
mídia só-leitura, host de gestão separado ou repositório assinado

A base guardada na própria máquina é editável por quem comprometeu a máquina. O invasor com `root` troca o binário, roda `aide --update`, e a verificação seguinte dá tudo em ordem. A base tem de viver fora do alcance do sistema monitorado — e a verificação ideal roda **de fora dele**, porque um *rootkit* de kernel mente para qualquer programa que rode ali dentro.

Monitore o que muda pouco e vale muito: binários, `/etc`, módulos de kernel, pontos de arranque automático (unidades, `cron`, `~/.bashrc`) e credenciais (`shadow`, `sudoers`, `authorized_keys`). **Deixe de fora o volátil por natureza** — `/var/log`, `/var/cache`, `/tmp`, `/proc` —, senão o ruído afoga o sinal.

DOIS REGISTROS QUE RESPONDEM PERGUNTAS DIFERENTES

AUDITD — O QUE O KERNEL OBSERVOU

evidência: independe da boa vontade do programa

- **Vigiar caminho:** `-w /etc/sudoers -p wa -k escalada` (`w`=escrita, `a`=atributo, `-k`=chave de consulta).
- **Vigiar chamada:** `-a always,exit -S execve -F euid=0`. Meça o volume antes de deixar fixa.
- **Persistir:** `/etc/audit/rules.d/*.rules` + `augenrules --load`. Via `auditctl`, só na memória.
- **`-e 2` torna o conjunto imutável até o boot** — nem `root` altera as regras. É o que vira evidência.

JOURNALD — O QUE O PROGRAMA CONTOU

diagnóstico: rico, mas voluntário

- Registra o que a aplicação **decidiu** registrar — se ela não conta, não há linha.
- Ótimo para entender **por que** um serviço falhou; fraco como prova do que alguém fez.
- **Log que vive só na máquina monitorada é prova sob custódia do suspeito.** Envie para fora: `systemd-journal-remote`, `audisp-remote`.
- `chattr +a` num log impede reescrever e truncar — a limpeza seletiva de rastro deixa de funcionar.

`uid=0 auid=1001`

O campo mais valioso de um evento. O `auid` é fixado no *login* e **imutável dali em diante**: esta linha diz que foi o usuário **1001** quem, via sudo, fez aquilo como root. Numa máquina com cinco administradores, é a diferença entre "alguém apagou" e "fulano apagou".

NA PRÁTICA — E O QUE CADA CAMADA REALMENTE COBRE

kali@kali: ~

```
# 1. os hashes que o apt já mantém — 30 segundos
$ sudo debsums -c
/usr/bin/whoami            ← um byte a mais e ele aparece
# 2. auditoria: regras, e depois torná-las imutáveis
$ sudo auditctl -w /etc/shadow -p wa -k identidade
$ sudo augenrules --load && sudo auditctl -l
$ sudo auditctl -e 2   # imutável até o reboot
$ sudo ausearch -k identidade -ts today -i
$ sudo ausearch -m USER_LOGIN -sv no # logins que falharam
# 3. linha de base explícita — e a custódia dela
$ sudo aideinit && sudo aide --check
$ sudo sha256sum /var/lib/aide/aide.db  # leve para fora
# 4. o que nem root altera sem dar um passo auditável
$ sudo chattr +i /etc/passwd    # imutável
$ sudo chattr +a /var/log/importante # só acréscimo
```

CAMADA	COBRE — E O BURACO QUE DEIXA
`debsums -c`	tudo que veio de pacote, sem configurar nada. Não vê o que foi instalado fora do `apt`
`aide --check`	o que você declarar, com hash e metadados. Custa manter a base fora da máquina
`auditd`	**quem** fez, com qual chamada. Volume alto se a regra for genérica
`chattr +i` / `+a`	não impede `root`: obriga um passo a mais, e auditável
log remoto	tira a prova da custódia do suspeito — exige um coletor de pé

Quando o alerta é real: não desligue (memória e conexões morrem junto), isole da rede sem apagar nada, colete antes de mexer (`ss -tunap`, `ps auxf`, `lsof -n`, `last`), preserve com hash — e **não conserte por cima**: sistema comprometido em nível de `root` só volta a ser confiável reconstruído de mídia limpa.

`rkhunter` e `chkrootkit` **rodam sobre o sistema que investigam.** Um *rootkit* em nível de kernel responde o que quiser a quem perguntar — e essas ferramentas perguntam. E só detectam o **conhecido**. Diante de suspeita séria, a análise correta é feita **de fora**: disco montado em outra máquina, ou o *live* USB do Kali em **modo forense**, que não monta nada automaticamente nem toca na área de troca.

RESUMO RÁPIDO

- ✓ **Detecção exige uma linha de base registrada quando o sistema estava íntegro.** É a única etapa que não se faz em retrospecto.
- ✓ **Base guardada na máquina monitorada é adulterável por quem a comprometeu** — a custódia externa é o que separa o método do teatro.
- ✓ `auditd` **registra o que o kernel observou;** `journald`**, o que os programas decidiram contar.** O `auid` responde "quem fez", não só "com qual UID".
- ✓ `-e 2` **torna as regras imutáveis até o reboot** — e o reboot é ele próprio um evento visível.
- ✓ `debsums -c` **é a verificação de trinta segundos**; o AIDE cobre o resto, ao custo de manter a base.
- ✓ **Log que só existe na máquina monitorada não é prova.** Envie para fora no momento em que é gerado.

NO KALI

O Kali *rolling* é hostil a uma linha de base ingênua — é a lição mais prática do capítulo aqui. Num Debian estável `/usr/bin` fica parado por meses e uma mudança é notável; no Kali, um `full-upgrade` semanal reescreve centenas de binários e o `aide --check` seguinte devolve milhares de linhas — em duas semanas o operador aprende a ignorar o relatório, que é **o modo mais comum de um sistema de detecção falhar**. O método que funciona: `aide --update` logo após cada atualização planejada, **com a base nova recustodiada fora da máquina**. Assim qualquer mudança fora da janela de atualização vira anomalia por definição. **A máquina Kali é a mais interessante de auditar da sua rede e não é a mais monitorada:** concentra credencial de laboratório, captura e resultado de teste autorizado — e ao mesmo tempo um `auditd` mal configurado nela gera ruído gigantesco. Comece pelas regras de identidade e escalada e deixe o `execve` genérico de fora até medir o volume. `debsums -c` **é o teste que você pode rodar hoje:** a resposta esperada é lista vazia, e qualquer binário de `/usr/bin` que apareça merece explicação imediata. **Perfil de MAC bem construído é sensor:** um perfil que gera zero negações em operação normal transforma *qualquer* `apparmor="DENIED"` em evento — cruzado com o `auditd`, dá duas fontes independentes. **RHEL:** o `auditd` é nativo e já vem habilitado, com conjuntos alinhados a normas em `/usr/share/audit/sample-rules/` — `30-stig.rules` dá um ponto de partida auditado. A verificação por pacote é `rpm -Va`.